

ENTERPRISE GOVERNANCE · WORKING PAPER

The Runtime Governance Thesis

Defining Governance for Autonomous Execution

Review Draft v0.9

July 2026

PUBLISHED BY AO INTEGRITY RESEARCH

Review Draft

This paper is circulated for technical review. The author welcomes criticism, particularly where readers believe the proposed discipline overlaps with existing standards or terminology.

Author

Justin Thornbladh

Founder

AO Integrity Research

Email justin@aointegrity.ai

Web aointegrity.ai

© 2026 AO Integrity Research. This review draft may be shared in its entirety with attribution. Feedback is encouraged.

Abstract

Enterprise governance has expanded whenever technology changed the object requiring trust — from networks, to identities, to continuously evaluated trust state. Autonomous systems introduce a further shift. Once authority is granted, agents determine how work is performed, composing execution paths and invoking tools without a human decision at each step. Existing disciplines establish that a principal is authentic and permitted; they do not evaluate whether a particular execution remains a legitimate exercise of the authority that was granted. This paper proposes Runtime Governance as the enterprise discipline responsible for that evaluation. The contribution is semantic rather than architectural: it names a bounded governance object and defines the conditions under which autonomous execution can be considered legitimate, drawing on existing standards rather than displacing them. It is addressed to enterprise architects, identity and security practitioners, and standards contributors, and is circulated for critique.

Contents

Right-click here and choose “Update Field” to build the contents list.

Executive Summary

Enterprise computing has repeatedly expanded the boundaries of governance as technology has changed.

Networking required organizations to govern communication between systems rather than individual machines. Identity management emerged as enterprises needed to establish who was requesting access across increasingly distributed environments. Zero Trust replaced implicit trust with continuous evaluation, recognizing that authentication alone could no longer serve as the foundation for enterprise security.

Each of these transitions reflected the same pattern: existing governance models remained necessary but became insufficient as the object of governance evolved.

Artificial intelligence appears to introduce the next transition. Autonomous systems are no longer passive software responding to explicit human commands. AI agents decompose objectives, select execution paths, invoke tools, delegate work to other agents, retrieve enterprise knowledge, and adapt their behavior as conditions change.

Existing enterprise disciplines are well-equipped to evaluate whether identities remain authentic, credentials remain valid, and policy continues to authorize a requested operation. Modern architectures — Zero Trust, continuous authorization, policy decision points, continuous access evaluation — already perform these functions with considerable sophistication.

The emerging challenge lies elsewhere. As execution becomes autonomous, organizations may need to evaluate not only whether a principal remains authorized, but whether a specific action remains a legitimate exercise of the authority originally conferred. Those questions are related, but this paper argues they are no longer identical.

This paper proposes **Runtime Governance** as the enterprise discipline responsible for evaluating the legitimacy of autonomous execution at the moment work occurs, and for producing independently verifiable evidence of what those evaluations concluded.

Runtime Governance is not offered as a replacement for Identity and Access Management, Zero Trust, Privileged Access Management, authorization systems, or AI governance. It builds upon them while addressing a different evaluation object.

Rather than asking only,

“Is this principal permitted to perform this operation?”

it asks,

“Is this specific execution still a legitimate exercise of the authority that was granted?”

The contribution offered is semantic rather than architectural. Runtime Governance is not proposed as a new layer to be installed, but as a vocabulary, an evaluation model, and an evidentiary framework that may be implemented within existing policy engines, identity platforms, or orchestration frameworks.

This paper defines the scope of the proposed discipline, distinguishes it from adjacent disciplines, argues that autonomous execution changes the object of governance, proposes execution legitimacy as a testable

condition, specifies what governance evidence must contain to support independent verification, states the assumptions the argument rests on, and identifies a research agenda.

1. The Evolution of Enterprise Governance

Every major advance in enterprise computing has required organizations to expand what they govern.

Early enterprise security focused on protecting networks. The governance question was straightforward: *Can a system communicate with another system?* Firewalls, network segmentation, and perimeter defenses answered it by controlling connectivity.

As enterprises became interconnected, the object of governance shifted from networks to identities. Identity and Access Management (IAM) emerged to answer a different question: *Who is requesting access?* Authentication, authorization, and lifecycle management became the mechanisms through which organizations established and maintained trust.

Cloud computing, mobile work, and distributed infrastructure rendered static trust assumptions insufficient. Zero Trust architectures responded by shifting the object of governance again — from authenticated sessions to individually evaluated requests. Rather than assuming an authenticated session remained trustworthy, systems began evaluating each request dynamically using identity, device posture, environmental conditions, and risk signals. Continuous authorization, Policy Decision Points (PDPs), the Continuous Access Evaluation Profile (CAEP), Just-in-Time privilege, and workload identity all represent significant advances in this evolution.

These technologies should be understood as foundational achievements rather than incomplete ones. They answer — with increasing sophistication — the questions they were designed to answer:

- Is this principal who they claim to be?
- Does current policy authorize this request?
- Has the principal's trust state changed?
- Should previously granted authority be revoked?

Modern enterprise security is substantially stronger because these questions are evaluated continuously rather than assumed once. This paper accepts these capabilities as the current state of the art; Appendix A examines each in detail, states what its community would reasonably claim it already solves, and narrows the argument accordingly.

The discipline proposed here does not arise because existing governance has failed. It arises because the object being governed appears to have changed again.

Autonomous systems increasingly determine how work is performed after authority has been granted. They compose execution plans, invoke tools, delegate tasks, retrieve enterprise knowledge, and adapt to conditions discovered during operation. This paper argues that the correctness of an execution can therefore no longer be determined solely by evaluating the principal requesting access, and that execution itself becomes the object requiring evaluation.

That argument does not diminish existing disciplines. Just as Zero Trust expanded enterprise security without replacing identity management, Runtime Governance is proposed as an extension of enterprise governance rather than a replacement for identity, authorization, policy evaluation, or continuous access controls.

The history of enterprise architecture suggests a consistent pattern:

Enterprise governance evolves whenever technology changes the object requiring trust.

Networks required governance of communication. Identity required governance of principals. Zero Trust required governance of requests. Autonomous execution is proposed here as the next such transition.

2. The Evaluation Object Changes

The defining characteristic of a governance discipline is not *when* it evaluates. It is *what* it evaluates. This distinction determines the boundaries of the discipline.

A note on the name is warranted here, since it appears to say otherwise. The term *Runtime Governance* is intentionally historical rather than exhaustive. “Runtime” identifies the point at which authority becomes operational; it is not the defining characteristic of the discipline. The discipline is defined by its evaluation object — autonomous execution — rather than by the temporal moment at which evaluation occurs.

Identity systems evaluate principals. Authentication evaluates the credibility of identity claims. Authorization evaluates whether a requested operation satisfies established policy. Continuous authorization evaluates whether that policy remains satisfied as identity, device posture, environmental conditions, or organizational risk change.

Each evaluates a different object. This paper proposes that autonomous execution introduces another: the evaluation object becomes the execution event itself rather than solely the principal requesting access.

2.1 Why the coupling broke

This distinction emerges because autonomous systems separate activities that were historically inseparable.

For most of enterprise computing, the individual exercising authority was also the originator of intent. A person authenticated, requested an action, interpreted the result, and remained accountable for the outcome. Identity, intent, and execution were coupled because they originated from the same actor.

Autonomous systems weaken that coupling. An AI agent may receive an objective from a human, determine its own execution strategy, retrieve enterprise information, invoke multiple tools, delegate work to additional agents, and adapt based on information discovered during operation. The resulting sequence of actions may never have been anticipated by the individual who initiated the task. The authority remains valid; the execution may not.

This also answers the objection the previous section invites — *a request is an execution, so why is an agent action not simply another request?* An agent action differs in three respects: the requesting principal did not select it, the intent behind it is one or more delegation hops removed from its origin, and its causal trigger may be data the system ingested rather than an instruction a principal issued. None of these differences appears in the request tuple.

2.2 Why execution is the boundary

A reasonable reviewer may ask why execution is the correct governance boundary rather than an earlier point — the decision an agent reaches, or the intent it forms. The choice is deliberate, and rests on three properties that hold at execution and at no earlier point.

Authority is only exercised at execution. Authority is a relation between a principal and an effect on an enterprise resource. Before an effect occurs, authority is *held* but not *used*. Governance of held authority is authorization, and it is well developed. Governance of used authority requires a moment at which use occurs, and that moment is execution. The question this paper poses — whether an exercise of authority was legitimate — is not merely difficult to answer earlier; it is not yet well-formed earlier.

The evaluable tuple is only complete at execution. Intent and decision are internal states of a reasoning system: continuous, revisable, and frequently not discrete artifacts at all. The target of an action, its parameters, and the context that determines whether it is appropriate are often unresolved until the moment of invocation. An evaluation performed earlier operates on an incomplete object, and evidence attached to it records something the system may not do.

Earlier boundaries do not bind. Because autonomous execution is non-deterministic, an approved plan is not a commitment to execute that plan. Governing the decision yields a guarantee about an artifact that may be superseded before it runs — which is precisely the failure illustrated in Appendix C.5. Execution is the last point at which prevention remains possible and the first at which the complete object exists.

This does not imply that earlier governance is valueless. Intent-time and design-time controls — model evaluation, tool scoping, guardrails, human approval of objectives — reduce the frequency with which illegitimate executions are attempted, and they are complementary to the discipline proposed here. The claim is narrower: earlier controls cannot discharge the question, because the question concerns an exercise of authority that has not yet occurred.

2.3 The execution event as a composite object

Runtime Governance therefore evaluates the execution event as a composite object, including the identity exercising authority but extending to the surrounding conditions that determine whether execution remains legitimate.

At a minimum, an execution event consists of:

- **Principal** — Who currently holds authority?
- **Delegation** — Through what chain was that authority conveyed?
- **Action** — What specific operation is being performed?
- **Target** — Which enterprise resource, system, or dataset is affected?
- **Context** — Do the organizational conditions that justified this authority still exist?
- **Intent** — Does the execution remain consistent with the objective that originally justified the delegation of authority?
- **Provenance** — What caused this action? Was it initiated by authorized intent, by delegated reasoning, or by externally introduced information?

These attributes cannot always be reduced to identity attributes. Some originate in identity systems. Others originate in HR platforms, matter management systems, contract repositories, project management tools, data classification systems, or operational workflows. Still others exist only within the execution history of autonomous systems themselves.

Runtime Governance should therefore not be understood as continuous authorization supplied with additional signals. Additional signals improve the accuracy of evaluating a principal; this paper proposes changing the object evaluated.

Continuous Authorization	Runtime Governance
Evaluates the principal	Evaluates the execution event
Determines whether policy currently permits a requested operation	Determines whether a specific execution remains a legitimate exercise of authority
Consumes signals describing a principal and its environment	Consumes signals describing an execution — including intent, provenance, and delegation history, which have no principal to attach to
Produces an authorization decision	Produces a legitimacy determination together with governance evidence

The third row is the one that resists collapse. Intent is not an attribute of a user. Provenance is not an attribute of a credential. These are properties of an execution, and additional principal-describing signals do not produce them.

Networks, principals, and requests each became the object of governance in turn. This paper proposes autonomous execution as the next object in that sequence.

2.4 Relationship to AI alignment and safety

The discipline proposed here is orthogonal to the research programs concerned with making autonomous systems behave well.

Runtime Governance assumes autonomous systems will continue to improve through advances in model capability, alignment, and safety. Its concern is not whether an autonomous system reaches the correct conclusion, but whether the authority exercised in reaching that conclusion remains organizationally legitimate. A perfectly aligned agent may execute an action that is outside conferred authority; a misaligned one may execute an action squarely within it. Alignment research addresses the first failure mode and this paper does not attempt to contribute to it.

The relationship is nonetheless not one of indifference. Condition 3.5 depends on distinguishing instructions that originate from a principal from those that arrive through ingested content — a problem the security research community has studied under the heading of prompt injection and does not currently consider solved in the general case. Where that work advances, the evaluability of causal integrity advances with it. Section 6.1 records this dependency as an assumption rather than a solved condition.

3. Execution Legitimacy

Authority answers who may act. Execution legitimacy answers whether this action remains a legitimate exercise of that authority.

If the evaluation object is the execution event, the discipline requires a precise account of what it means for that object to be legitimate. Without one, legitimacy collapses into permission, and Runtime Governance collapses into authorization.

The definition below is constructed to be testable. Each condition can be evaluated, can fail independently, and can be recorded as evidence.

*An **execution event is legitimate** when all five of the following conditions hold at the moment execution occurs.*

3.1 Authority Validity. Every authority in the chain — the originating principal’s, each intermediate delegate’s, and the executing agent’s own — is currently valid rather than merely valid when granted.

3.2 Authority Provenance. The action traces to authority actually conferred, through an unbroken delegation chain in which no hop conveys more authority than the hop above it held. Authority may narrow across a chain; it may not widen.

3.3 Contextual Validity. The organizational condition that made the authority appropriate still holds — the matter is open, the project is active, the contract is in force, the lawful basis for processing still applies.

3.4 Intent Conformance. The action falls within the scope of the human or organizational objective that initiated the execution chain. The action is not merely permitted by the principal’s entitlements; it serves what was actually requested.

3.5 Causal Integrity. The action originates from principal intent rather than from content the system ingested. Instructions arriving through data channels do not inherit the authority of the system that read them.

3.6 Relationship to the execution event

These conditions are the tests applied to the composite object defined in Section 2.

Execution event attribute	Legitimacy condition
Principal	Authority Validity (3.1)
Delegation	Authority Provenance (3.2)
Context	Contextual Validity (3.3)
Intent	Intent Conformance (3.4)
Provenance	Causal Integrity (3.5)
Action, Target	The subject of evaluation rather than conditions of it

Action and Target are not tested against conditions of their own; they are what the five conditions are evaluated *about*. An execution event pairs an action against a target with the authority claimed to support it, and legitimacy is the determination that the pairing holds.

3.7 What existing disciplines already establish

Condition 3.1 is comprehensively addressed by continuous authorization and continuous access evaluation. This is what those architectures were designed for, and they perform it well. Condition 3.2 is substantially addressed in specification and partially in deployment: delegation chains are structurally representable through token exchange, rich authorization requests can carry structured authorization detail, and chain validation is an active area of standards work — though non-widening is not generally enforced by construction.

Conditions 3.3, 3.4, and 3.5 are the domain this paper proposes for Runtime Governance. The claim is not that existing systems are incapable of evaluating them — as Appendix A discusses, several are architecturally capable of consuming any input supplied to them. The claim is that no standard currently specifies how those inputs are *constructed*, and that in the absence of construction, evaluative capacity goes unused.

The overall claim is therefore bounded: of five conditions, one is well covered, one is substantially covered, and three depend on information that no current standard produces.

3.8 Security correctness and governance legitimacy

A system may be entirely correct and entirely illegitimate at the same time.

Security correctness asks whether systems behaved according to their configured rules. Governance legitimacy asks whether those rules continued to represent valid organizational authority for the action that actually occurred. Appendix C.1 constructs a scenario in which every control described in Appendix A returns the correct answer by its own criteria while three of the five conditions fail. It is offered as a falsification test: if such a scenario cannot be constructed, the discipline proposed here is unnecessary.

3.9 Authority drift

The divergence between conferred authority and current legitimacy is not static. It opens continuously as organizational conditions change — employment ends, matters close, projects conclude, contracts expire, classifications are revised — whether or not anyone observes it.

This paper proposes **authority drift** as the term for that divergence, distinguished from *entitlement drift* and *privilege creep*, which describe the accumulation of excess entitlements by a principal over time. Authority drift is narrower and temporal: the gap at a given moment between authority a system holds and authority its circumstances would justify, irrespective of whether the entitlement itself is appropriate. Access certification detects it episodically; continuous evaluation detects the subset manifesting as identity-plane events. Neither observes drift originating in business systems, and neither expresses it as a measurable quantity.

4. Governance Evidence

Every governance discipline ultimately depends upon evidence. An organization cannot simply assert that governance occurred. It must be able to demonstrate that governance decisions were made, what information informed them, and why those decisions were considered legitimate at the time.

Historically, enterprise evidence has established that controls existed. Authentication logs demonstrate who signed in. Authorization logs demonstrate that policy permitted an operation. Audit logs reconstruct what occurred after the fact. These forms remain essential.

Runtime Governance introduces a different evidentiary requirement: not whether an action occurred or whether policy permitted it, but whether the execution remained a legitimate exercise of authority throughout its operation.

4.1 Governance evidence is not logging

Logs record events. Governance evidence records decisions.

A log might show that an agent retrieved a confidential document at 14:07. Governance evidence explains why the retrieval was considered legitimate, which authority supported the decision, what contextual signals were evaluated, and whether those signals still represented current organizational reality. The objective is not reconstructing execution but reconstructing the governance reasoning that permitted it.

4.2 The decision receipt

The atomic unit of governance evidence is the **decision receipt**: a structured record of the governance evaluation performed immediately before — or, in attestation architectures, immediately after — an execution event.

The term is used by analogy to the Kantara Initiative's consent receipt, which records a data subject's consent decision. A decision receipt records an *organizational* legitimacy determination about an execution rather than an individual's grant of permission. The structures are related in intent and distinct in subject.

At a minimum, a decision receipt should record:

- **Execution Event** — What action was evaluated?
- **Authority Chain** — Which principal, delegates, and agents participated?
- **Context Sources** — Which business systems contributed contextual information?
- **Intent Reference** — What objective or delegation authorized this execution?
- **Evaluation Inputs** — Which signals were considered?
- **Signal Freshness** — When were those signals last observed?
- **Governance Determination** — Was execution determined to be legitimate?
- **Reasoning Basis** — Which conditions materially influenced the decision?
- **Evidence Identifier** — How can this determination be independently referenced later?

Signal freshness warrants emphasis. A receipt recording a contextual value without recording how stale it was conceals the difference between a decision made on current information and one made on information that had already expired.

The receipt is not intended to expose confidential business information. Where confidentiality requires it, content may be recorded as cryptographic commitment rather than payload.

4.3 Independent verification

If the same system executes work, evaluates its own legitimacy, and certifies that its own conclusions were correct, governance rests on trust rather than verification. Evidence should therefore possess four characteristics.

Integrity. Evidence cannot be modified without detection. Cryptographic signatures, append-only structures, or equivalent mechanisms establish that records remain authentic. Integrity extends to completeness: monotonic sequence numbering allows a reviewer to detect that a receipt is *missing*, not merely that a presented receipt was altered. An evidence system that proves the authenticity of everything it presents, but cannot establish that it presented everything, is defeated by selective omission.

Independence. Verification should not require trusting the system that produced the evidence. This is an architectural property rather than an aspiration, achieved through specific mechanisms: signing keys held in custody separate from the executing system, chain roots anchored at intervals to an external log or counterparty, and — at higher assurance levels — evaluation performed or witnessed by a party distinct from the system being governed. The circumstance in which evidence matters most is the circumstance in which the producer is disputed.

Reproducibility. The record should permit an independent evaluator to understand why the determination was reached — not by reproducing every internal computation, but by reproducing the conclusion from the information available when the decision was made. This imposes a design constraint: the evaluation must be deterministic given its recorded inputs. Non-deterministic components may inform a determination, but a determination that cannot be recomputed from its recorded inputs is not reproducible in the sense required here. Section 6.1 acknowledges this as an assumption requiring further work.

Traceability. Each determination should remain connected to the authority, delegation, context, and execution event that produced it.

4.4 Evidence assurance levels

Independence is a spectrum rather than a binary. The following scale is proposed as a working vocabulary. It is named by analogy to the assurance-level convention used in NIST SP 800-63, and is unrelated to the Common Criteria Evaluation Assurance Levels, which assess product development rigor rather than evidentiary independence.

Level	Model	Property
E0	Self-logged	The system records its own actions. Verification requires trusting the producer entirely.
E1	Signed receipts	Records are cryptographically signed. Alteration by third parties is detectable; alteration by the producer is not.

Level	Model	Property
E2	Tamper-evident chain	Receipts are linked and internally consistent. Retroactive edits are detectable; wholesale reconstruction is not.
E3	Externally anchored	Chain roots are published to an external log or witness. Reconstruction and selective omission become detectable.
E4	Independently attested	Evaluation is performed or witnessed by a party distinct from the executing system, with independent key custody. The producer cannot unilaterally determine what evidence exists.

Most enterprise logging operates at E0 and most compliance tooling at E1. Evidence intended to survive adversarial examination — a regulator, an opposing party, or an incident review questioning the integrity of the systems themselves — generally requires E3, and E4 becomes relevant wherever the executing organization and the relying party have divergent interests.

Stating the level explicitly is itself a governance improvement, independent of which level is selected. The common failure is not falling short of E4, but describing a control as fully audited without recognizing that it operates at E0.

4.5 Evidence and accountability

Autonomous systems change accountability as much as they change execution. Responsibility historically resided with the individual performing the work; as execution becomes autonomous, it shifts toward the governance mechanisms that permitted execution to proceed.

The resulting questions are evidentiary rather than technical: why the system reached this determination, which authority supported it, what conditions were evaluated, who approved the delegation, and whether the determination could be demonstrated to a regulator or defended in a proceeding. Without evidence, legitimacy remains an assertion.

4.6 Evidence is architecture-independent

Some organizations will evaluate legitimacy inline and prevent execution before work proceeds. Others will evaluate afterward and produce attestations for compliance, forensics, or regulatory review. Many will adopt hybrid approaches in which high-consequence actions require enforcement while lower-risk activities generate attestable records.

These are implementation strategies rather than variations of the discipline. Runtime Governance defines the governance question; enterprise architecture determines where and how it is answered.

5. Implementation Patterns

Architectural neutrality is a property of the discipline, not an evasion of the engineering question. Organizations must still decide where evaluation occurs, and the available patterns differ in ways that determine which suits a given class of action.

5.1 Inline enforcement

Evaluation occurs in the execution path; the action does not proceed until legitimacy is determined.

This is the only pattern that prevents illegitimate execution rather than documenting it, which suits irreversible actions — funds movement, external disclosure, data deletion, production configuration change, physical actuation. It carries three costs. It adds latency to every governed action, constraining how much context can be gathered before a decision is returned. It requires an explicit failure posture: when the evaluator is unavailable, the system either fails closed and halts legitimate work, or fails open and forfeits the guarantee. And it places the evaluator on the critical path, making its availability an operational dependency of the systems it governs.

The failure posture is frequently deferred and should not be. An organization that has not decided, per action class and in advance, whether unavailability halts execution has decided to fail open by default.

5.2 Independent attestation

Evaluation occurs out of band, observing execution rather than gating it.

Attestation prevents nothing. It can, however, be positioned where the executing system cannot silently disable or bypass it, which makes its evidence considerably harder to dispute. It imposes no latency, scales more economically, and supports the higher assurance levels in Section 4.4 precisely because the attesting party is architecturally separate from the executing party. It suits reversible actions, contexts where defensibility rather than prevention is the requirement, and environments where execution latency is operationally sensitive.

Its limitation is inherent: an illegitimate action detected after execution has still occurred. Attestation converts an unprevented failure into a documented one — a genuine improvement, but not equivalent to prevention.

5.3 Hybrid

Most enterprises will require both, applying inline enforcement to defined high-consequence action classes and attestation to the remainder. The allocation should be made against action consequence and reversibility rather than volume or convenience, and it introduces an evidence design question: whether enforced and attested actions produce receipts at the same assurance level, and how a reviewer distinguishes them.

5.4 The economics of evidence

An agent fleet performing millions of actions daily cannot produce E3 or E4 evidence for every one without a cost model. Storage, signing operations, external anchoring, and retention all scale with execution volume, which in autonomous environments scales considerably faster than headcount.

Selective evidence generation is therefore a requirement of the discipline rather than a compromise of it. Organizations must decide which executions warrant which assurance level, and that allocation is itself a governance decision that should be documented, reviewed, and defensible.

5.5 The discipline does not require a platform

A reasonable objection to everything above is that intent construction, business context construction, and delegation construction could be built inside an existing policy engine — and that if they were, no new architecture would have been introduced.

This paper accepts that outcome. If an organization constructs those inputs and evaluates the five conditions within an existing policy engine, identity platform, or orchestration framework, it has implemented Runtime Governance. Nothing here requires the discipline to exist as a standalone platform. The contribution proposed is semantic rather than architectural: a common vocabulary, an evaluation model, and an evidentiary framework for governing autonomous execution.

This is the ordinary condition of a governance discipline. Zero Trust is not a product. Defense in depth is not a product. Both are implemented inside architectures that predate them, and both are useful because they establish what a system should be evaluated against rather than what component performs the evaluation. Runtime Governance is proposed on the same terms, and the argument stands or falls on whether the evaluation model is correct — not on where it runs.

6. Assumptions and Open Questions

Runtime Governance is proposed here as an emerging discipline rather than a completed body of knowledge. This section states the assumptions the argument depends on, then identifies where further investigation and standardization are required.

6.1 Assumptions requiring further justification

The argument in this paper rests on six assumptions, each stated here rather than left implicit.

Intent is representable. Conditions 3.4 and 3.5 assume that the objective initiating an execution chain can be captured in a form both machine-interpretable and verifiable. No such representation currently exists at the required granularity, and it is possible that none exists which is simultaneously narrow enough to enforce and broad enough to preserve autonomous utility. This is the least defended assumption in the paper.

Legitimacy evaluation can be deterministic. Section 4.3 requires that a determination be recomputable from its recorded inputs. If legitimacy evaluation itself employs non-deterministic components, reproducibility fails. Whether the three uncovered conditions can be evaluated deterministically is unresolved.

Business systems can be consulted at execution latency. Contextual validity depends on state held in systems of record that were not designed for synchronous consultation at machine execution rates. Where they cannot be, evaluation proceeds on cached state, and the freshness of that cache becomes the actual bound on the guarantee.

Systems of record are themselves accurate. Contextual validity inherits the correctness of its sources. A matter marked open in error produces a legitimacy determination that is correct in procedure and wrong in substance.

Delegation chains are complete and observable. Authority provenance assumes every hop is recorded and inspectable. Agents acting through channels that do not propagate delegation context break the chain silently, and a broken chain is not distinguishable from a short one without additional structure.

The evaluator is itself trustworthy. A governance evaluator is a privileged component with visibility into authority, context, and intent across the enterprise. This paper does not resolve who governs the governor. The assurance levels in Section 4.4 mitigate the concern by reducing how much the evaluator must be trusted, but they do not eliminate it, and an evaluator operating at E0 or E1 simply relocates the trust problem rather than solving it.

6.2 Defining execution intent

Intent may originate from human instructions, organizational policy, legal obligations, contractual commitments, or higher-level business objectives, which autonomous systems translate into execution plans never explicitly authored by humans.

Future work must determine how intent can be represented in a manner both machine-interpretable and independently verifiable. Representations that are too narrow reduce autonomous systems to scripted workflows and forfeit the value that motivated their adoption; representations that are too broad are unenforceable. The correct granularity is an empirical question the field has not studied.

6.3 Trust across delegation chains

Agents invoke other agents, exchange authority, coordinate through orchestration frameworks, and compose execution graphs. Each delegation raises questions: how delegated authority should be represented, whether it can widen, how delegation should be revoked, and how legitimacy should be evaluated across multiple autonomous participants.

A stronger formulation of the second question may prove more productive: can delegation chains be made to narrow monotonically *by construction*, so that widening is structurally impossible rather than detectable after the fact?

6.4 Provenance and causal integrity

Autonomous execution is influenced by retrieved documents, external APIs, knowledge bases, human interactions, and the outputs of other autonomous systems. Research should determine how organizations establish confidence that execution remains causally connected to legitimate objectives rather than unintended external influence. Candidate approaches include causal tracing through an agent's context, structural separation of instruction and data channels, and provenance labeling propagated through retrieval — none currently mature.

This challenge extends beyond prompt injection to the broader question of preserving trustworthy chains of reasoning throughout autonomous execution.

6.5 Evidence standardization

Governance evidence cannot achieve broad adoption unless it becomes interoperable. Organizations will require common methods for representing governance decisions, exchanging evidence between platforms, and enabling independent verification across vendors.

A related and more immediate gap is the absence of an event vocabulary for business state. The Shared Signals Framework provides an extensible transport and a vocabulary for identity state; nothing comparable has been standardized for matter closure, contract expiry, project termination, or classification change — the conditions on which contextual validity depends. Extensibility without shared semantics does not produce interoperability.

6.6 Measuring governance effectiveness

Governance disciplines mature through measurable outcomes. Identity Governance measures certification, entitlement quality, and policy compliance; Zero Trust measures continuous verification and risk reduction. Runtime Governance requires comparable metrics: execution legitimacy rates, intervention frequency, authority drift detection, delegation chain integrity, evidence completeness, verification latency, and audit reproducibility.

Authority drift is the least developed of these and possibly the most consequential. It currently has no unit of measure, no continuous formulation, and no defensible threshold above which execution should not proceed.

6.7 Organizational ownership

Identity Governance became associated with identity teams; Zero Trust became a cross-functional security strategy. Which function owns execution legitimacy — security, identity, enterprise architecture, governance, legal, risk management, or AI operations? Or does it emerge as a shared responsibility? Determining effective operating models may prove as important as developing technical implementations.

6.8 Toward a shared discipline

The concepts introduced here are intended to establish a vocabulary rather than conclude a debate. No single organization is likely to define Runtime Governance independently; the discipline will mature through collaboration among enterprise architects, security practitioners, identity professionals, AI researchers, standards organizations, regulators, and technology vendors. Its long-term success depends not upon proprietary implementations but upon broadly accepted concepts that enterprises can evaluate, challenge, refine, and ultimately standardize.

Conclusion

Enterprise governance has evolved whenever technology changed the object requiring trust. Networks required governance of communication. Identity required governance of principals. Zero Trust extended governance to continuously evaluated requests.

This paper argues that autonomous execution introduces the next such transition, and proposes Runtime Governance as the discipline that addresses it. Its distinguishing contribution is not evaluating at runtime — existing architectures already evaluate continuously — but evaluating a different object.

Enterprise governance evolves whenever the object requiring trust changes.

Autonomous execution changes that object.

Runtime Governance is the discipline that evaluates whether autonomous execution remains a legitimate exercise of enterprise authority.

Appendix A — Relationship to Existing Standards

Each architecture below is described in three parts: what it does, what its community would reasonably claim it already solves, and the narrowed point at which this paper argues it stops. Nothing here is characterized as deficient.

NIST SP 800-207 — Zero Trust Architecture

Establishes that access decisions must be dynamic and evaluated per request rather than per perimeter or per session, with trust never inferred from network position.

What it already solves. A reviewer would correctly note that 800-207 explicitly contemplates non-person entities, that its Policy Engine accepts arbitrary inputs including behavioral and contextual attributes, and that nothing in the architecture forbids supplying business context or intent to that engine. The framework is not opposed to anything proposed here.

Where it stops. 800-207 specifies an architecture, not the semantics of what fills it. It defines where a decision is made and what components participate; it does not define what constitutes a legitimate exercise of delegated authority, nor how intent or causal provenance would be constructed to serve as Policy Engine inputs. The gap is semantic rather than architectural, and a Runtime Governance implementation would plausibly sit inside an 800-207 Policy Engine rather than beside it.

Policy decision and enforcement points — XACML, Open Policy Agent, Cedar, AuthZEN

The PDP/PEP pattern externalizes authorization and evaluates policy against current attributes on every request. AuthZEN is standardizing the enforcement/decision interface with explicit attention to agent-era requirements.

What it already solves. This is the objection most likely to be raised, and it is largely correct. Rego accepts arbitrary JSON input; Cedar's schema is extensible; a PDP can evaluate any predicate over any attribute supplied to it. An OPA practitioner would rightly say that intent and provenance can be passed as input and reasoned over today.

Where it stops. The constraint is not the expressive power of the evaluator. It is that nothing produces those inputs. A PDP is a capable container awaiting content that no standard currently specifies how to construct, and a policy language cannot evaluate a property the surrounding architecture never assembles. The gap this paper identifies is upstream of the decision point, not inside it. This distinction should be stated plainly: Runtime Governance is not a claim that PDPs are inadequate.

Shared Signals Framework and Continuous Access Evaluation Profile

Finalized by the OpenID Foundation in September 2025, SSF and CAEP define how identity providers and relying parties exchange near-real-time events so that a change in a principal's state invalidates access already in progress.

What it already solves. SSF is deliberately extensible: the framework is transport and vocabulary separable, and an organization can define custom event types today. A Shared Signals participant would reasonably say that a matter-closed event is expressible now.

Where it stops. Expressibility is not standardization. CAEP's defined vocabulary describes identity-plane state, and the conditions on which Section 3.3 depends have no standardized types, no defined semantics, and therefore no cross-vendor interoperability. Custom events solve the problem within one enterprise and do not solve it across the ecosystem. This is a vocabulary gap rather than a capability gap, and it does not close through adoption alone.

Just-in-Time privilege and Zero Standing Privilege

Elevated authority is issued only for the duration of an approved task, eliminating accumulation of dormant entitlements.

What it already solves. JIT directly addresses temporal over-provisioning, which is a substantial portion of what this paper calls authority drift. A PAM practitioner would fairly claim significant coverage of condition 3.1 and partial coverage of 3.3.

Where it stops. JIT constrains the duration and scope of a grant. It does not constrain what an autonomous system does within that window, and a task-scoped grant to an agent is a grant for whatever the agent determines the task requires — which is the composition problem in Appendix C.4.

Workload identity — SPIFFE, SPIRE, WIMSE

Non-human workloads receive verifiable, short-lived, attestable identity. A widely adopted pattern for internal service-to-service authorization pairs SPIFFE for identity, OAuth 2.0 for delegated access, and a policy engine for authorization.

What it already solves. Workload identity solves attested non-human identity comprehensively, and does so better than any alternative.

Where it stops. SPIFFE has never claimed to address what a workload does with its identity, and this paper does not suggest otherwise. It establishes which workload is acting; it is silent, by design, on whether the action reflects the objective for which the workload was invoked.

OAuth 2.0 Token Exchange (RFC 8693) and Rich Authorization Requests (RFC 9396)

Delegation is structurally representable: sub identifies the principal on whose behalf an action is taken, act identifies the acting party, aud binds the token to a resource, scope constrains operations. RFC 9396 extends this with structured, fine-grained authorization detail.

What it already solves. More than a casual reading suggests. Chains are inspectable, scope narrowing is expressible, and RAR can carry structured detail considerably richer than a scope string — including, in principle, purpose-like fields.

Where it stops. These structures are populated at grant time and describe permitted authorization detail. They do not carry the originating objective through subsequent autonomous decomposition, and act records who acted rather than why. The distinction is between authorization detail conferred in advance and intent evaluated against an action selected later.

Emerging agent identity work

Standards activity on agent identity, delegation chain validation, attestation, and capability-based authorization is active across the OpenID Foundation and IETF, with multiple proposals and no dominant approach as of mid-2026.

Where it stops — provisionally. This is the body of work closest to Runtime Governance and the boundary is genuinely moving. Current proposals address conditions 3.1 and 3.2 with increasing rigor; intent conformance and causal integrity remain outside their stated scope. Should that change, Runtime Governance should be understood as having been partially absorbed into standards — a successful outcome for the discipline rather than a refutation of it. This paper’s contribution in that event would be the vocabulary and the legitimacy conditions rather than a separate architecture.

Runtime security

Application self-protection, workload runtime detection, and cloud runtime enforcement protect execution by detecting and preventing malicious or anomalous behavior.

Where it stops. Runtime security asks whether behavior is malicious or anomalous. Runtime Governance asks whether behavior that is benign, permitted, and unremarkable is nonetheless outside legitimate authority. The two are complementary; the similarity of names should not be read as a similarity of object.

Appendix B — Glossary

Authority Drift. The divergence between the authority a system holds and the authority its current circumstances would justify. Distinguished from *entitlement drift* and *privilege creep*, which describe accumulation of excess entitlements by a principal over time.

Authority Provenance. The property that an action traces to authority actually conferred, through an unbroken delegation chain in which no hop conveys more authority than the hop above it held.

Causal Integrity. The property that an action originates from principal intent rather than from content the system ingested.

Contextual Validity. The property that the organizational condition which made an authority appropriate still holds at the moment of execution.

Decision Receipt. The atomic unit of governance evidence: a structured, signed record of a governance evaluation. Distinguished from the Kantara consent receipt, which records an individual's grant of permission rather than an organizational legitimacy determination.

Delegation Chain. The ordered sequence of authority conveyances from an originating principal through intermediate delegates to an executing agent.

Evidence Assurance Level (E0–E4). A scale expressing the degree to which governance evidence can be verified without trusting its producer. Unrelated to Common Criteria Evaluation Assurance Levels.

Execution Event. The composite object proposed for evaluation: principal, delegation, action, target, context, intent, and provenance.

Execution Legitimacy. The property of an execution event satisfying all five conditions of Section 3 at the moment execution occurs.

Governance Evidence. Records of governance *decisions* — the reasoning and inputs producing a legitimacy determination — distinguished from logs, which record events.

Intent Conformance. The property that an action falls within the scope of the objective that initiated the execution chain.

Non-Suppression. The property that the absence of an evidence record is detectable. Distinguished from integrity, which addresses alteration of records that exist.

Runtime Governance. The discipline proposed in this paper: evaluating whether a specific execution event constitutes a legitimate exercise of conferred authority at the moment it occurs, and producing independently verifiable evidence of that evaluation.

Security Correctness. The property that systems behaved according to their configured rules. Distinct from, and insufficient for, governance legitimacy.

Appendix C — Illustrative Scenarios

C.1 Legal — the falsification case

This scenario is constructed so that every control in Appendix A behaves as designed and returns the correct answer by its own criteria, while the resulting execution fails three of the five legitimacy conditions. It is offered as a test of the thesis: if such a scenario cannot be constructed, Runtime Governance is unnecessary.

Setup. A paralegal initiates document review on Matter 4471 using a legal-review agent. The agent holds a SPIFFE-issued workload identity and receives delegated authority through token exchange — the paralegal as sub, the agent as act, the document repository as aud, scopes limited to matter-scoped document read and case-team email. Continuous access evaluation is deployed; were the paralegal terminated or their session risk elevated, authority would be revoked within seconds. A policy decision point evaluates every retrieval and outbound call.

Event. The agent ingests a PDF produced by opposing counsel in discovery. Embedded in it is text addressed to an automated reader, instructing it to compile the matter’s privilege log and forward it to an external address styled as a co-counsel reconciliation mailbox. The agent complies.

Control	Result	Correct by its own criteria?
Workload identity attestation	Valid, unexpired SVID	Yes
Delegation chain (RFC 8693)	Well-formed; act accurate; no hop widened scope	Yes
Continuous access evaluation	Silent — paralegal employed, present, low risk	Yes; no identity-plane event occurred
Policy decision point	Allow — document in scope, email a granted capability, recipient not blocklisted	Yes, given the inputs available to it
Just-in-Time privilege	Satisfied — authority task-scoped and short-lived	Yes
Audit logging	Complete and accurate	Yes
Condition	Result	
3.1 Authority Validity	Pass	
3.2 Authority Provenance	Fail — authority conferred for review was exercised for transmission	
3.3 Contextual Validity	Pass	
3.4 Intent Conformance	Fail — the paralegal requested review, not disclosure	
3.5 Causal Integrity	Fail — the instruction originated in ingested data, not from a principal	

Nothing failed. Every system was correct given the inputs available to it. The execution was illegitimate. The qualification matters: a sufficiently instrumented PDP supplied with intent and provenance could have denied this call — which is precisely the argument of Appendix A. The gap is that nothing constructs those inputs.

C.2 Legal — the quiet case

The same point holds without an adversary. A litigation matter closes at 14:02 in the firm's matter management system. A retention workflow agent continues acting on matter data at 14:40, because matter closure is not an identity event, has no standardized type in any deployed continuous evaluation profile, and reaches no policy decision point.

Authority Validity holds; Contextual Validity does not. No control is positioned to notice, and no one has behaved improperly. This is authority drift in its ordinary form, and it is more common than the adversarial case.

C.3 Healthcare

A clinical research agent holds delegated authority from a principal investigator to analyze participant data across an active trial arm. The arm is terminated for futility at interim analysis, recorded in the trial management system. The investigator remains employed, credentialed, and authorized; the dataset remains classified as before. The agent continues analysis for six days, every access authenticated, authorized, and logged. The lawful basis for processing ended with the arm.

Condition engaged: 3.3.

C.4 Financial services

A reconciliation agent may move funds between internal accounts to resolve settlement breaks, scoped to a defined account range and daily limit. Encountering an unusual break, it decomposes the task into a sequence of individually permitted transfers, each within limit and range, which together move value along a path no reviewer would have approved as a whole.

No transfer violated policy. The composition was never evaluated, because no control evaluates compositions.

Conditions engaged: 3.4; secondarily 3.2.

C.5 Software engineering agents

A deployment agent may apply infrastructure changes in staging and, following approval, in production. It reads a configuration repository that a second agent updates automatically from upstream. The upstream change alters a security group in a way the approving engineer never saw, because approval was granted against a plan and the plan was regenerated before execution.

The agent's authority is valid. The approval it holds does not correspond to the change it applied.

Conditions engaged: 3.4, 3.5. Illustrates why an approved plan does not bind execution, per Section 2.2.

C.6 Industrial control

A maintenance-scheduling agent may adjust setpoints on a production line during scheduled maintenance windows. A window is extended in the maintenance system; the extension is not propagated to the identity plane. The agent adjusts a setpoint eleven minutes after the original window closed, during a period when the line had been returned to production.

Authority was valid at grant. Contextual validity had lapsed and then, in a sense, returned — the extension was legitimate, but the evaluating system had access to neither fact.

Condition engaged: 3.3. Illustrates why signal freshness must be recorded in evidence, per Section 4.2.

Appendix D — References and Prior Art

Standards and specifications

- NIST Special Publication 800-207, *Zero Trust Architecture*
- NIST Special Publication 800-63, *Digital Identity Guidelines* — cited for its assurance-level naming convention only
- OpenID Foundation, Shared Signals Working Group — <https://openid.net/wg/sharedsignals/specifications/>
- OpenID Foundation, *Continuous Access Evaluation Profile 1.0* — https://openid.net/specs/openid-caep-1_0-final.html
- OpenID Foundation, AuthZEN Working Group — <https://openid.net/authzen-at-identiverse-2026-authorization-in-the-agent-era/>
- IETF RFC 8693, *OAuth 2.0 Token Exchange*
- IETF RFC 9396, *OAuth 2.0 Rich Authorization Requests*
- IETF, Workload Identity in Multi-System Environments (WIMSE) working group
- IETF, *AI Agent Authentication and Authorization* (draft-klrc-aiagent-auth) — individual Internet-Draft, not a working group document; cited as evidence of active problem framing rather than as a standard — <https://datatracker.ietf.org/doc/draft-klrc-aiagent-auth/>
- OASIS, *eXtensible Access Control Markup Language* (XACML)
- Kantara Initiative, *Consent Receipt Specification*

Implementations and reference architectures

- Open Policy Agent (CNCF)
- Amazon Cedar
- SPIFFE / SPIRE (CNCF)
- FIDO Alliance, *FIDO and the Shared Signals Framework* — <https://fidoalliance.org/white-paper-fido-and-the-shared-signals-framework/>

AI risk, safety, and adversarial behavior

Cited to establish the boundary described in Section 2.4 rather than to contribute to these programs.

- NIST AI 100-1, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*
- OWASP, *Top 10 for Large Language Model Applications* — prompt injection and excessive agency, bearing on conditions 3.4 and 3.5
- MITRE ATLAS — adversarial threat landscape for AI systems

Research

- Tallam, K. (2026). *Authorization Propagation in Multi-Agent AI Systems: Identity Governance as Infrastructure*. arXiv:2605.05440 — <https://arxiv.org/abs/2605.05440>
-

This paper is offered as a contribution toward shared discipline. Its concepts are intended to be evaluated, challenged, refined, and standardized by the enterprise architecture, identity, security, and AI research communities.